

Speech Recognition

Exercises 04 about Regular Expressions in Python

Prof. Dr.-Ing. Udo Garmann

Prerequisites

Before you start the following exercises, learn about Python's regular expressions.

On python.org you find the following introductions:

<https://docs.python.org/3/library/re.html>

<https://docs.python.org/3/howto/regex.html>

Interactive Tools: <https://regex101.com/>

In Python RegEx can be used in several ways: Compile one and use a RegEx-string. Compiled RegEx are useful if you want to apply the same RegEx several times, e.g. in a loop.

In this exercise we need the functions `match()`, `sub()`, ...

Theory

Part 01

Give at least 1 example and 1 counter-example of the full matches of the following REs:

```
r 'TH(D|I|N|R) '
r 'TH(D|I|N|R)+'
r 'TH\s'
r '[1-9][0-9]+'
r '([\^a-zA-Z]) [tT] he([\^a-zA-Z]) '
r '\b[0-9]+(\.[0-9]+)? *(km|Kk)ilometers?\b'
```

Part 02

Find regular expressions for the following types of strings:

- Phone numbers city codes including country prefix, e.g. +42 001
- E-Mail addresses, e.g. test@test.de
- Mathematical expression of a fraction that look like 23 / 42 (number operator number)
- An case-sensitive identifier in a programming language that looks like this: Start with a character from a to z or A to Z and continue with an arbitrary sequence of a character like the starting one or a decimal number. No other characters are allowed.

Part 03

Create a list that contains at least the following words:

```
['wanted', 'learned', 'justify', 'justified', 'read', 'red', 'get', 'feed', 'shed',
'embed', 'proceed']
```

Write a program that applies a regular expression on each word. The RE must remove a -ed suffix from a word. The output must be a new list with the changed words.

In order to make the program compact use lists comprehensive, see e.g. https://www.w3schools.com/python/python_lists_comprehensive

Practical Part

Part 01

In the current version of the chatbot example (code_02_chatbot.py) no intent recognition is done and the only response is a greeting, see code line 165.

Extend the chatbot so that it separates the greeting intent from other intents. Choose as other intent the get_weather one.

Analyse the prompt text with regular expressions. Specify at least 3 different formulations for a greeting and create one RegEx to recognize them. The greetings must be case-sensitive and at least 2 of them must consist of 2 words.

Take care and quote characters in the RegEx that are also used in the Python syntax, e.g. ' becomes \'.

Part 02

After the greeting ask for a name and store it in a streamlit session var.

The user input after the chatbot's name request must have the form / starts with "my name is ...". The name consists of the last 2 words in input, last is lastname, before is first name (first- and lastname).

<https://docs.python.org/3/library/re.html#match-objects>

Part 03

After the name input there the chatbot asks for an e-mail address. The user input simply is the address. It must be checked by the chatbot and - when correct - stored in a session variable.

This functionality of getting specific values for some parameters is called Slot Filling. Other examples would be to store a city like Deggendorf in a location slot or a date slot like "2026-04-13".